AI Reflective Statement

Group Members:

Yash Nadkarni H00410472

Gunasai Anumulapally H00557945

Shyam Agarwal H00553801

Manav Parmar H00552843

1. Introduction

Our group used OpenAI's ChatGPT to support the development, debugging and evaluation of our neural-network-based inverse kinematics coursework. We chose ChatGPT because it could provide programming assistance, explain robotics concepts and help troubleshoot problems through an iterative conversation.

2. Use of Generative AI

One significant use was developing python code to automate repeated neural network hyperparameter experiments, especially the ones whose data had been overwritten and lost. ChatGPT helped create scripts for testing different architectures, activation functions, learning rates and input features, allowing us to compare configurations systematically rather than manually modfying and executing each experiment.

We also used ChatGPT to implement neural network based controllers in Webots. These controllers loaded our trained models, processed the target sphere's coordinates, predicted joint values, applied clipping and denormalization, and commanded Ned and SCARA to follow the target.

Additionally, ChatGPT generated a motor-control function for our Webots evaluation. This function used position sensors, a tolerance threshold and consecutive successful checks to determine whether the robot had settled before measuring TCP error.

ChatGPT was also used to make helper functions for normalization, denormalization, evaluating ned's joint 1 region and finding the top 5 worst samples.

3. Prompting Approach

Our prompts were generally specific. We provided existing code, explained the intended behaviour and requested modifications compatible with our implementation. For example, we asked ChatGPT to adapt our analytical inverse kinematics sphere controller to use trained neural network predictions through an entirely new file.

We also requested explanations of technical concepts, comparisons between network configurations and debugging assistance when implementation problems occurred. This was crucial in helping us identify the joint region problems, for which horizontal radial distance as the 6th input was used.

4. Checking and Verification

We did not assume AI-generated code was automatically correct. We executed the suggested scripts, inspected their outputs and compared the results against our coursework requirements.

For the neural network controllers, we tested both robots using multiple target positions and monitored their joint commands and TCP tracking errors, to see if the robot arms would actually follow the sphere or not.

The motor-settling function was checked through Webots position sensors to ensure that measurements were taken after movement had stabilized.

We also reviewed normalization ranges, robot identifiers and joint limits to ensure compatibility between training and simulation.

5. Reflection on Learning

ChatGPT helped us better understand the connection between forward kinematics, neural network regression and inverse kinematics prediction. Its explanations also supported our understanding of how network architecture, dataset size and feature engineering influenced performance.

However, ChtGPT's suggestions still required critical examination. For example, joint-limit assumptions and model file paths needed adjustment to match our final implementation. We learned that generated code can be technically plausible while still being unsuitable for a particular robot configuration.

6. Final Statement

Overall, ChatGPT was useful for accelerating experimentation, supporting debugging and clarifying technical concepts. Nevertheless, our group remained responsible for testing the implementation, interpreting results and preparing the final coursework deliverables. Our main lesson was that generative AI is most effective when its suggestions are treated as starting points requiring independent verification.

Appendix (Example Prompts)

Prompt 1: Hyperparameter Experiments

"Modify our neural-network training code to automate experiments with different hidden-layer configurations, activation functions, learning rates and input features. Record the validation results for both Ned and SCARA so we can compare their performance. Also save them as a dataset file."

Prompt 2: Webots Neural-Network Controller

"Similar to the existing analytical inverse-kinematics sphere controller, create a version that loads our trained shared neural-network model and predicts the joint values required for Ned to follow the red target sphere. Adapt the same approach for SCARA."

Prompt 3: Motor Settling and Evaluation

"Create a Webots function in python that moves the robot to predicted joint positions and waits until the position sensors confirm that the joints have reached their targets before measuring TCP error.“